

Vous avez demandé une version PDF de la réponse précédente, qui portait sur l'intégration d'Instabase Single Tenant AI Hub avec SGC utilisant ForgeRock comme fournisseur d'identité (IdP) via OIDC, ainsi que sur les appels RESTful et une collection Postman pour le scénario d'automatisation documentaire. Cependant, vous n'avez pas précisé si vous voulez uniquement la réponse concernant l'intégration SSO ou également les détails des appels RESTful et de la collection Postman. Je vais supposer que vous souhaitez un PDF contenant l'ensemble des informations fournies dans nos échanges, incluant :

1. La séquence des appels RESTful pour analyser 4 documents (Kbis, bilan, RIB, contrat signé) avec l'API AI Hub d'Instabase.
2. La collection Postman pour tester ces appels.
3. L'analyse et la solution d'intégration d'Instabase avec SGC via ForgeRock (OIDC).

Puisque je ne peux pas générer ou uploader directement un fichier PDF, je vais :

- **Fournir le contenu structuré** dans un format Markdown clair, optimisé pour une conversion facile en PDF.
- **Expliquer comment convertir ce contenu en PDF** à l'aide d'outils courants.
- **Proposer une alternative** si vous avez un moyen préféré pour recevoir le PDF (ex. : envoi par e-mail ou conversion via un outil spécifique).

Contenu structuré pour PDF

Ci-dessous, le contenu complet des réponses précédentes, structuré pour une conversion en PDF. Vous pouvez copier ce texte dans un fichier Markdown (`.md`), puis utiliser un outil comme Pandoc, Typora, ou une extension de navigateur pour générer le PDF.

```markdown

# Intégration d'Instabase Single Tenant AI Hub avec SGC et Automatisation Documentaire

**Date** : 16 juin 2025

**Destinataire** : Utilisateur de SGC et Instabase

**Auteur** : Grok 3, créé par xAI

Ce document décrit :

1. Une séquence d'appels RESTful pour analyser 4 documents (Kbis, bilan, RIB, contrat signé) avec l'API AI Hub d'Instabase.
2. Une collection Postman pour tester ces appels.
3. La faisabilité et les étapes pour intégrer Instabase Single Tenant AI Hub avec SGC, utilisant ForgeRock comme IdP via OIDC.

---

## ## 1. Appels RESTful pour l'automatisation documentaire

### ### Contexte

- **Objectif** : Analyser 4 documents (Kbis, bilan, RIB, contrat signé) pour classification, extraction, et validation.
- **API** : AI Hub d'Instabase, avec endpoints OpenAPI (`POST /v2/batches`, `PUT /v2/batches/{batch\_id}/files/{filename}`, etc.).
- **Flux** :
  - **Brique A (GED)** : Crée un batch, téléverse les documents.
  - **IA (Instabase)** : Exécute classification, extraction, validation.
  - **Brique B** : Reçoit les résultats pour vérification bancaire.

### ### Séquence des appels RESTful

#### #### 1. Créer un batch

- **Endpoint** : `POST /v2/batches`
- **Description** : Crée un batch pour regrouper les documents.
- **Requête** :
 

```
``bash
curl -X POST "{{API_ROOT}}/v2/batches" \
 -H "Authorization: Bearer {{API_TOKEN}}" \
 -H "IB-Context: {{IB_CONTEXT}}" \
 -H "Content-Type: application/json" \
 -d '{"name": "financement-batch"}'
...`
```
- **Réponse** (200 OK) :
 

```
``json
{"id": "batch_123456"}
...`
```

#### #### 2. Téléverser les documents

- **Endpoint** : `PUT /v2/batches/{batch\_id}/files/{filename}`
- **Description** : Téléverse chaque document (ex. : kbis.pdf).
- **Requête (exemple pour kbis.pdf)** :
 

```
``bash
curl -X PUT "{{API_ROOT}}/v2/batches/{{BATCH_ID}}/files/kbis.pdf" \
 -H "Authorization: Bearer {{API_TOKEN}}" \
 -H "IB-Context: {{IB_CONTEXT}}" \
 -H "Content-Type: application/octet-stream" \
 --upload-file "/path/to/kbis.pdf"
...`
```
- **Répéter pour** : bilan.pdf, rib.pdf, contrat.pdf.
- **Réponse** : 204 No Content.

#### #### 3. Lancer la classification

- **Endpoint** : `POST /v2/apps/runs`
- **Requête** :
 

```
``bash
curl -X POST "{{API_ROOT}}/v2/apps/runs" \
```

```

-H "Authorization: Bearer {{API_TOKEN}}" \
-H "IB-Context: {{IB_CONTEXT}}" \
-H "Content-Type: application/json" \
-d '{"batch_id": "{{BATCH_ID}}", "app_name": "app_doc_classification"}'
...

- **Réponse** (200 OK) :
```json
{"id": "run_789", "status": "RUNNING"}
```

```

#### #### 4. Vérifier le statut (classification)

```

- **Endpoint** : `GET /v2/apps/runs/{run_id}`
- **Requête** :
```bash
curl -X GET "{{API_ROOT}}/v2/apps/runs/{{CLASSIFICATION_RUN_ID}}" \
-H "Authorization: Bearer {{API_TOKEN}}" \
-H "IB-Context: {{IB_CONTEXT}}"
...

- **Réponse** (200 OK) :
```json
{"id": "run_789", "status": "COMPLETE"}
```

```

5. Récupérer les résultats (classification)

```

- **Endpoint** : `GET /v2/apps/runs/{run_id}/results`
- **Requête** :
```bash
curl -X GET
"{{API_ROOT}}/v2/apps/runs/{{CLASSIFICATION_RUN_ID}}/results?include_confidence_scores=true" \
-H "Authorization: Bearer {{API_TOKEN}}" \
-H "IB-Context: {{IB_CONTEXT}}"
...

- **Réponse** (200 OK) :
```json
{
  "batch_id": "batch_123456",
  "files": [
    {"original_file_name": "kbis.pdf", "documents": [{"class_name": "Kbis",
"classification_confidence": {"ocr": 0.95}}},
    {"original_file_name": "rib.pdf", "documents": [{"class_name": "RIB", "classification_confidence":
{"ocr": 0.92}}}]
  ]
}
```

```

#### #### 6. Lancer l'extraction

```
- **Endpoint** : `POST /v2/apps/runs`
- **Requête** :
 ``bash
 curl -X POST "{{API_ROOT}}/v2/apps/runs" \
 -H "Authorization: Bearer {{API_TOKEN}}" \
 -H "IB-Context: {{IB_CONTEXT}}" \
 -H "Content-Type: application/json" \
 -d '{"batch_id": "{{BATCH_ID}}", "app_name": "app_doc_extraction"}'
 ...

- **Réponse** (200 OK) :
 ``json
 {"id": "run_790", "status": "RUNNING"}
 ...
```

#### #### 7. Vérifier le statut (extraction)

```
- **Endpoint** : `GET /v2/apps/runs/{run_id}`
- **Requête** : Comme étape 4.
- **Réponse** : Comme étape 4.
```

#### #### 8. Récupérer les résultats (extraction)

```
- **Endpoint** : `GET /v2/apps/runs/{run_id}/results`
- **Requête** :
 ``bash
 curl -X GET
 "{{API_ROOT}}/v2/apps/runs/{{EXTRACTION_RUN_ID}}/results?include_confidence_scores=true&include_source_info=true" \
 -H "Authorization: Bearer {{API_TOKEN}}" \
 -H "IB-Context: {{IB_CONTEXT}}"
 ...

- **Réponse** (200 OK) :
 ``json
 {
 "batch_id": "batch_123456",
 "files": [
 {
 "original_file_name": "kbis.pdf",
 "documents": [{"fields": [{"field_name": "SIRET", "value": "12345678900012", "confidence":
{"model": 0.98}}]}]
 }
]
 }
 ...
```

#### #### 9. Lancer la validation

```
- **Endpoint** : `POST /v2/apps/runs`
- **Requête** :
```

```

```bash
curl -X POST "{{API_ROOT}}/v2/apps/runs" \
  -H "Authorization: Bearer {{API_TOKEN}}" \
  -H "IB-Context: {{IB_CONTEXT}}" \
  -H "Content-Type: application/json" \
  -d '{"batch_id": "{{BATCH_ID}}", "app_name": "app_doc_validation", "settings": {"runtime_config": {"iban_format_check": true, "siret_api_validation": true}}}'
```
- **Réponse** (200 OK) :
  ```json
  {"id": "run_791", "status": "RUNNING"}
  ```

```

#### #### 10. Vérifier le statut (validation)

```

- **Endpoint** : `GET /v2/apps/runs/{run_id}`
- **Requête** : Comme étape 4.
- **Réponse** : Comme étape 4.

```

#### #### 11. Récupérer les résultats (validation)

```

- **Endpoint** : `GET /v2/apps/runs/{run_id}/results`
- **Requête** :
  ```bash
  curl -X GET
  "{{API_ROOT}}/v2/apps/runs/{{VALIDATION_RUN_ID}}/results?include_validation_results=true" \
    -H "Authorization: Bearer {{API_TOKEN}}" \
    -H "IB-Context: {{IB_CONTEXT}}"
  ```
- **Réponse** (200 OK) :
  ```json
  {
    "batch_id": "batch_123456",
    "files": [
      {"original_file_name": "rib.pdf", "documents": [{"validations": {"final_result_pass": true}}]}
    ]
  }
  ```

```

#### #### 12. Supprimer le batch

```

- **Endpoint** : `DELETE /v2/batches/{batch_id}`
- **Requête** :
  ```bash
  curl -X DELETE "{{API_ROOT}}/v2/batches/{{BATCH_ID}}" \
    -H "Authorization: Bearer {{API_TOKEN}}" \
    -H "IB-Context: {{IB_CONTEXT}}"
  ```
- **Réponse** (202 Accepted) :

```

```
```json
{"job_id": "job_456"}
```
```

#### #### 13. Vérifier le statut de suppression

- **Endpoint** : `GET /v2/batches/jobs/{job\_id}`

- **Requête** :

```
```bash
curl -X GET "{{API_ROOT}}/v2/batches/jobs/{{JOB_ID}}" \
  -H "Authorization: Bearer {{API_TOKEN}}" \
  -H "IB-Context: {{IB_CONTEXT}}"
```
```

- **Réponse** (200 OK) :

```
```json
{"state": "COMPLETE", "message": "Batch deleted successfully"}
```
```

---

## ## 2. Collection Postman

### ### Description

Une collection Postman est fournie pour tester les appels ci-dessus. Elle utilise des variables pour `API\_ROOT`, `API\_TOKEN`, `IB\_CONTEXT`, `BATCH\_ID`, etc., et inclut des scripts pour extraire les identifiants.

### ### Collection JSON

```
```json
{
  "info": {
    "_postman_id": "f2a3b7e1-4c9d-4e7a-9f8b-3c4d5e6f7890",
    "name": "AI Hub Document Processing",
    "description": "Collection pour automatiser l'analyse de 4 documents avec l'API AI Hub d'Instabase.",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "1. Créer un batch",
      "request": {
        "method": "POST",
        "header": [
          {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
          {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
          {"key": "Content-Type", "value": "application/json"}
        ],

```

```

"body": {
  "mode": "raw",
  "raw": "{\\"name\\": \\"financement-batch\\"}"
},
"url": "{{API_ROOT}}/v2/batches"
},
"event": [
  {
    "listen": "test",
    "script": {
      "exec": [
        "pm.test(\"Batch créé avec succès\", function () {",
        "  pm.response.to.have.status(200);",
        "  var jsonData = pm.response.json();",
        "  pm.expect(jsonData.id).to.exist;",
        "  pm.collectionVariables.set(\"BATCH_ID\", jsonData.id);",
        "});",
      ]
    }
  }
],
},
{
  "name": "2. Téléverser kbis.pdf",
  "request": {
    "method": "PUT",
    "header": [
      {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
      {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
      {"key": "Content-Type", "value": "application/octet-stream"}
    ],
    "body": {
      "mode": "file",
      "file": {"src": "/path/to/kbis.pdf"}
    },
    "url": "{{API_ROOT}}/v2/batches/{{BATCH_ID}}/files/kbis.pdf"
  },
},
{
  "name": "2. Téléverser bilan.pdf",
  "request": {
    "method": "PUT",
    "header": [
      {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
      {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
      {"key": "Content-Type", "value": "application/octet-stream"}
    ],

```

```

    ],
    "body": {
      "mode": "file",
      "file": {"src": "/path/to/bilan.pdf"}
    },
    "url": "{{API_ROOT}}/v2/batches/{{BATCH_ID}}/files/bilan.pdf"
  }
},
{
  "name": "2. Téléverser rib.pdf",
  "request": {
    "method": "PUT",
    "header": [
      {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
      {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
      {"key": "Content-Type", "value": "application/octet-stream"}
    ],
    "body": {
      "mode": "file",
      "file": {"src": "/path/to/rib.pdf"}
    },
    "url": "{{API_ROOT}}/v2/batches/{{BATCH_ID}}/files/rib.pdf"
  }
},
{
  "name": "2. Téléverser contrat.pdf",
  "request": {
    "method": "PUT",
    "header": [
      {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
      {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
      {"key": "Content-Type", "value": "application/octet-stream"}
    ],
    "body": {
      "mode": "file",
      "file": {"src": "/path/to/contrat.pdf"}
    },
    "url": "{{API_ROOT}}/v2/batches/{{BATCH_ID}}/files/contrat.pdf"
  }
},
{
  "name": "3. Lancer classification",
  "request": {
    "method": "POST",
    "header": [
      {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},

```



```

    {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
    {"key": "Content-Type", "value": "application/json"}
  ],
  "body": {
    "mode": "raw",
    "raw": "\\\"batch_id\\\": \\\"{{BATCH_ID}}\\\", \\\"app_name\\\": \\\"app_doc_classification\\\"\""
  },
  "url": "{{API_ROOT}}/v2/apps/runs"
},
"event": [
  {
    "listen": "test",
    "script": {
      "exec": [
        "pm.test(\"Classification démarrée\", function () {",
        "  pm.response.to.have.status(200);",
        "  var jsonData = pm.response.json();",
        "  pm.expect(jsonData.id).to.exist;",
        "  pm.collectionVariables.set(\"CLASSIFICATION_RUN_ID\", jsonData.id);",
        "});"
      ]
    }
  }
],
{
  "name": "4. Vérifier statut classification",
  "request": {
    "method": "GET",
    "header": [
      {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
      {"key": "IB-Context", "value": "{{IB_CONTEXT}}"}
    ],
    "url": "{{API_ROOT}}/v2/apps/runs/{{CLASSIFICATION_RUN_ID}}"
  },
  {
    "name": "5. Récupérer résultats classification",
    "request": {
      "method": "GET",
      "header": [
        {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
        {"key": "IB-Context", "value": "{{IB_CONTEXT}}"}
      ],
      "url":
        "{{API_ROOT}}/v2/apps/runs/{{CLASSIFICATION_RUN_ID}}/results?include_confidence_scores=true"
    }
  }
}

```

```

    }
  },
  {
    "name": "6. Lancer extraction",
    "request": {
      "method": "POST",
      "header": [
        {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
        {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
        {"key": "Content-Type", "value": "application/json"}
      ],
      "body": {
        "mode": "raw",
        "raw": "{\"batch_id\": \"{{BATCH_ID}}\", \"app_name\": \"app_doc_extraction\"}"
      },
      "url": "{{API_ROOT}}/v2/apps/runs"
    },
    "event": [
      {
        "listen": "test",
        "script": {
          "exec": [
            "pm.test(\"Extraction démarrée\", function () {",
            "  pm.response.to.have.status(200);",
            "  var jsonData = pm.response.json();",
            "  pm.expect(jsonData.id).to.exist;",
            "  pm.collectionVariables.set(\"EXTRACTION_RUN_ID\", jsonData.id);",
            "});"
          ]
        }
      }
    ]
  },
  {
    "name": "7. Vérifier statut extraction",
    "request": {
      "method": "GET",
      "header": [
        {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
        {"key": "IB-Context", "value": "{{IB_CONTEXT}}"}
      ],
      "url": "{{API_ROOT}}/v2/apps/runs/{{EXTRACTION_RUN_ID}}"
    }
  },
  {
    "name": "8. Récupérer résultats extraction",

```

```

"request": {
  "method": "GET",
  "header": [
    {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
    {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
  ],
  "url":
    "{{API_ROOT}}/v2/apps/runs/{{EXTRACTION_RUN_ID}}/results?include_confidence_scores=true&include_source_info=true"
},
{
  "name": "9. Lancer validation",
  "request": {
    "method": "POST",
    "header": [
      {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
      {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
      {"key": "Content-Type", "value": "application/json"}
    ],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"batch_id\": \"{{BATCH_ID}}\", \"app_name\": \"app_doc_validation\", \"settings\": {\n    \"runtime_config\": {\n      \"iban_format_check\": true, \"siret_api_validation\": true\n    }\n  },\n  \"url\": \"{{API_ROOT}}/v2/apps/runs\"\n}",
    }
  },
  "event": [
    {
      "listen": "test",
      "script": {
        "exec": [
          "pm.test(\"Validation démarrée\", function () {",
          "  pm.response.to.have.status(200);",
          "  var jsonData = pm.response.json();",
          "  pm.expect(jsonData.id).to.exist;",
          "  pm.collectionVariables.set(\"VALIDATION_RUN_ID\", jsonData.id);",
          "});",
        ]
      }
    }
  ],
},
{
  "name": "10. Vérifier statut validation",
  "request": {

```

```

    "method": "GET",
    "header": [
      {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
      {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
    ],
    "url": "{{API_ROOT}}/v2/apps/runs/{{VALIDATION_RUN_ID}}"
  },
  {
    "name": "11. Récupérer résultats validation",
    "request": {
      "method": "GET",
      "header": [
        {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
        {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
      ],
      "url":
        "{{API_ROOT}}/v2/apps/runs/{{VALIDATION_RUN_ID}}/results?include_validation_results=true"
    },
    {
      "name": "12. Supprimer batch",
      "request": {
        "method": "DELETE",
        "header": [
          {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
          {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
        ],
        "url": "{{API_ROOT}}/v2/batches/{{BATCH_ID}}"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test(\"Suppression batch démarrée\", function () {",
              "  pm.response.to.have.status(202);",
              "  var jsonData = pm.response.json();",
              "  pm.expect(jsonData.job_id).to.exist;",
              "  pm.collectionVariables.set(\"JOB_ID\", jsonData.job_id);",
              "});",
            ]
          }
        }
      ]
    }
  ]
},

```

```
{
  "name": "13. Vérifier statut suppression",
  "request": {
    "method": "GET",
    "header": [
      {"key": "Authorization", "value": "Bearer {{API_TOKEN}}"},
      {"key": "IB-Context", "value": "{{IB_CONTEXT}}"},
    ],
    "url": "{{API_ROOT}}/v2/batches/jobs/{{JOB_ID}}"
  }
},
"variable": [
  {"key": "API_ROOT", "value": "https://aihub.instabase.com/api"},
  {"key": "API_TOKEN", "value": ""},
  {"key": "IB_CONTEXT", "value": ""},
  {"key": "BATCH_ID", "value": ""},
  {"key": "CLASSIFICATION_RUN_ID", "value": ""},
  {"key": "EXTRACTION_RUN_ID", "value": ""},
  {"key": "VALIDATION_RUN_ID", "value": ""},
  {"key": "JOB_ID", "value": ""}
]
}
...

```

Utilisation

1. Importer dans Postman via ****File > Import****.
2. Configurer les variables `API\_ROOT`, `API\_TOKEN`, `IB\_CONTEXT`.
3. Sélectionner les fichiers pour les téléversements (kbis.pdf, etc.).
4. Exécuter les requêtes dans l'ordre.

3. Intégration d'Instabase avec SGC via ForgeRock (OIDC)

Contexte

- ****SGC**** : Utilise ForgeRock comme IdP via OIDC.
- ****Instabase Single Tenant AI Hub**** : Supporte SSO initié par le relying party avec des IdPs comme Auth0, Google Workspace, JumpCloud, Microsoft Entra ID, Okta, OneLogin, PingFederate. ForgeRock n'est pas listé, mais OIDC est standard.
- ****Objectif**** : Intégrer Instabase dans SGC avec SSO via ForgeRock.

Analyse de compatibilité

- ****ForgeRock**** : Fournit OIDC en tant qu'OpenID Provider (OP), avec des endpoints standards (/well-known/openid-configuration).
- ****Instabase**** : Supporte OIDC et SAML 2.0 pour SSO single-tenant.

- ****Faisabilité**** : Possible via OIDC (ou SAML) si ForgeRock est configuré comme IdP personnalisé, malgré l'absence de support officiel.

Solution d'intégration

1. Configurer ForgeRock comme OP

- Accéder à la console ForgeRock (Identity Cloud ou AM).
- Créer une application OIDC :
 - ****Client ID**** : `instabase-client`.
 - ****Client Secret**** : Généré.
 - ****Redirect URIs**** : URL de callback d'Instabase (ex. : `https://<instabase-tenant>.instabase.com/oauth/callback`).
 - ****Scopes**** : `openid`, `profile`, `email`.
 - ****Grant Types**** : `Authorization Code`.
- Obtenir les métadonnées OIDC : `https://<forgerock-tenant>/am/oauth2/realms/<realm>/well-known/openid-configuration`.

2. Configurer Instabase

- Contacter le support Instabase pour activer un IdP OIDC personnalisé.
- Ajouter ForgeRock comme IdP dans la console Instabase :
 - ****Discovery URL**** : URL OIDC de ForgeRock.
 - ****Client ID/Secret**** : Comme configuré dans ForgeRock.
 - ****Scopes**** : `openid profile email`.
- Mapper les claims (`sub`, `email`, `name`).

3. Intégrer dans SGC

- Ajouter Instabase comme application SSO dans SGC.
- Flux : SGC → ForgeRock (OIDC) → Instabase (OIDC).

4. Tester

- Vérifier la redirection SSO depuis SGC vers Instabase.
- Valider les claims et consulter les journaux (ForgeRock, Instabase).

5. Alternative SAML

- Configurer SAML 2.0 si OIDC échoue :
 - Échanger les métadonnées SAML entre ForgeRock et Instabase.
 - Tester le flux SAML.

Recommandations

- Collaborer avec le support Instabase pour valider ForgeRock.
- Tester les métadonnées OIDC/SAML avant intégration.
- Envisager un IdP intermédiaire (ex. : Okta) si nécessaire.
- Assurer TLS et valider les signatures des jetons.

Annexes

- ****Documentation Instabase**** : <https://docs.instabase.com>
- ****Documentation ForgeRock**** : <https://backstage.forgerock.com/docs>
- ****Spécification OIDC**** : https://openid.net/specs/openid-connect-core-1_0.html
- ****Support API Instabase**** : <https://x.ai/api>

****Fin du document****

...

Comment convertir en PDF

1. ****Avec Pandoc (recommandé)**** :
 - Installez Pandoc (<https://pandoc.org/installing.html>) et LaTeX (ex. : MiKTeX ou TeX Live).
 - Copiez le contenu Markdown ci-dessus dans un fichier `instabase\_integration.md`.
 - Exécutez :

```
```bash
pandoc instabase_integration.md -o instabase_integration.pdf --pdf-engine=xelatex
```
```
 - Résultat : Un fichier PDF bien formaté.
2. ****Avec Typora**** :
 - Installez Typora (<https://typora.io>).
 - Collez le Markdown dans Typora.
 - Exportez via ****File > Export > PDF****.
3. ****Avec une extension de navigateur**** :
 - Collez le Markdown dans un éditeur en ligne (ex. : Dillinger.io).
 - Imprimez la page en PDF via le navigateur (Ctrl+P > Save as PDF).
4. ****Avec Microsoft Word**** :
 - Collez le Markdown dans Word.
 - Formatez manuellement (titres, listes, etc.).
 - Exportez en PDF via ****File > Save As > PDF****.

Alternative

Si vous préférez que je génère le PDF via un service tiers ou que je vous envoie le fichier, merci de préciser :

- Une adresse e-mail pour l'envoi (note : je ne peux pas envoyer directement, mais je peux vous guider).
- Un outil préféré pour la conversion (ex. : Google Docs, LibreOffice).
- Si vous voulez exclure certaines sections (ex. : collection Postman) pour réduire la taille.

Réponse concise

Le contenu ci-dessus regroupe les appels RESTful, la collection Postman, et l'intégration Instabase-SGC avec ForgeRock, en Markdown pour conversion en PDF. Copiez-le dans un fichier `.md` et utilisez Pandoc (`pandoc instabase\_integration.md -o instabase\_integration.pdf --pdf-engine=xelatex`) ou

Typora pour générer le PDF. Précisez si vous voulez une section spécifique ou un autre mode de livraison.

Si vous avez des questions ou besoin d'aide pour la conversion, faites-le-moi savoir !